Appli Fil rouge STEAM (v1)

Le moins d'IA possible

Organisation

- Groupe de 2
- A déposer sur MyGES, fichier README.md
  - language utilisé (go, python, rust...)
  - base de données utilisée (mongodb, postgresql, ...)
  - dépot Github (obligatoirement, privé à partager avec `rcarlier`)

Objectifs

- une base de données issue du fichier json fourni https://data.atontour.info/steam.json.zip
- une API Rest codée selon des principes SOLID
- provisoirement stockage des users admin dans un fichier `users.json` (mdp haché) que l'on crée en cli `./webapi –u admin –p 'password'` (avec un meilleur mdp)
- fichier de tests (`test.http` , postman...)

setup

- Docker (ou Podman) pour MongoDB ou PostreSQL
- Langage de prog (Go, Python, Rust...)
- intégration des données steam (attention à convertir si PostreSQL - fournir un fichier de conversion)
- `.env` pour infos MONGO, nom du fichier user, JWT secrets...
- `example.env` idem, sans les infos sensibles
- `.gitignore`
- ...

Accès en lecture

- GET /health (vérifier la cnx mongodb)

- GET api/v1/games = liste des jeux
- GET api/v1/games/1 = jeu avec id 1

Pagination et tri

- GET api/v1/games?limit=2&skip=1
- GET api/v1/games?ordering=title
- GET api/v1/games?ordering=-title

Recherche

- GET api/v1/games?title=subnautica
- GET api/v1/games?release_year=2013
- GET api/v1/games?platforms=linux
- GET api/v1/games?genre=2,25

La liste doit retourner, par exemple, ce genre d'infos:


```
{
    "items": [
        /* les données */
    ],
    "page": 1,
    "pageSize": 10,
    "totalItems": 51823,
    "totalPages": 5183,
    "_links": {
        "self": "/api/v1/games?limit=10&skip=0",
        "next": "/api/v1/games?limit=10&skip=10"
    }
}
```


Accès protégés (JWT)

- POST api/v1/login
- POST api/v1/games
- PUT api/v1/games/1
- DELETE api/v1/games/1

Bonus (importants)

- doc: swagger, openAPI...
- logs
- CORS
- Format d’erreur cohérent avec les bons codes de statut
- readme.md (installation, exemples de requêtes)